

Spring & Spring Cloud — interview Q&A

EPAM round two prep | Satish

1 Spring core

What is a bean?

Any object Spring creates and manages, instead of you writing new.

Without Spring

```
OrderRepository repo = new OrderRepository();    // without Spring
PaymentClient client = new PaymentClient();      // every caller repeats
OrderService svc = new OrderService(repo, client);
```

With Spring

```
@Service
public class OrderService {
    private final OrderRepository repo;
    public OrderService(OrderRepository repo) {    // Spring passes it in
        this.repo = repo;
    }
}
```

Two ways to declare one: annotate the class (@Service, @Repository, @Component, @Controller — same mechanism, different names), or write a @Bean method for classes you do not own, like ThreadPoolTaskExecutor.

How does Spring core work internally?

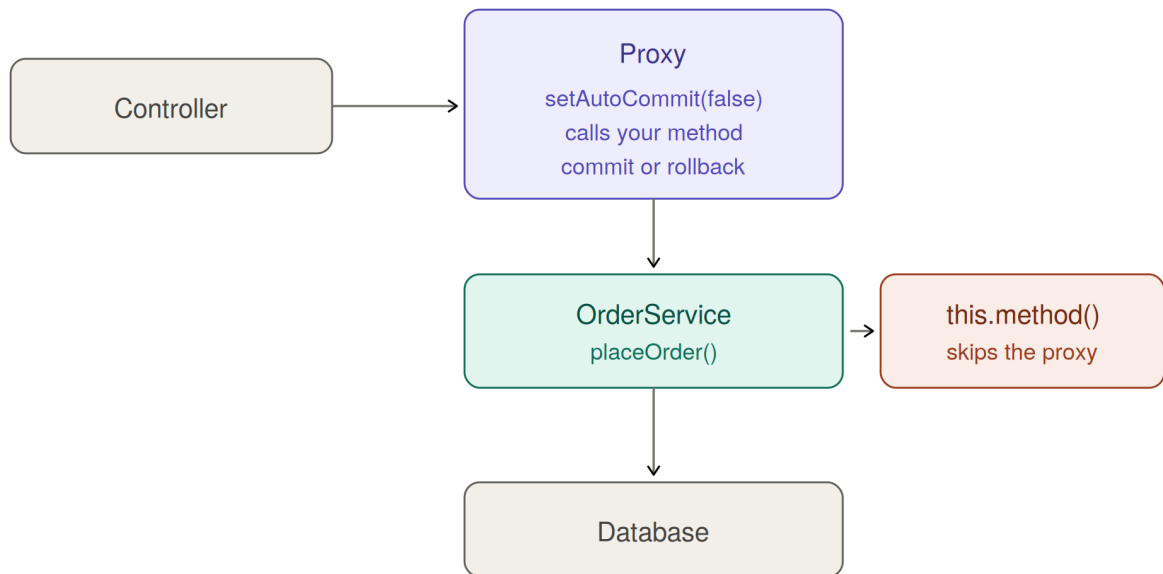
At startup Spring scans for annotations, builds a graph of what depends on what, instantiates in dependency order, and injects. That is DI; the registry is the IoC container. Everything else in Spring sits on top of this.

Key default: beans are **singletons** — one instance shared application-wide. That is why every class injecting the same executor gets the same pool.

How does @Transactional work internally?

The database runs COMMIT and ROLLBACK, but it has no idea your method threw. Something in the application must catch it and decide. Spring injects a **proxy** — a generated subclass wrapping yours — holding the try/catch. Same mechanism as FeignClient and all AOP.

```
// what the proxy effectively does
setAutoCommit(false);
try { super.placeOrder(order); commit(); }
catch (RuntimeException e) { rollback(); throw e; }
```



The controller never holds your bean — it holds the proxy

Three consequences they test:

- Self-invocation silently loses the transaction. An internal `this.placeOrder()` bypasses the proxy — no error, no rollback.
- `@Transactional` on a private method does nothing; it cannot be overridden.
- Only **unchecked** exceptions roll back by default. Checked exceptions commit. Override with `rollbackFor = Exception.class`.

2 Service registry (Eureka)

What is Eureka and how is it configured?

Eureka Server is a **deployable Spring Boot app** with its own port and dashboard. Eureka Client is a library inside each service.

Eureka server

```
@SpringBootApplication
@EnableEurekaServer // this is the whole server
public class EurekaServerApplication { }
```

Eureka server — application.yml

```
server:
  port: 8761

eureka:
  client:
    register-with-eureka: false # the server does not register with itself
    fetch-registry: false
```

Each service — application.yml

```
spring:
  application:
    name: order-service                # the name everyone looks it up by

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
  instance:
    lease-renewal-interval-in-seconds: 30    # heartbeat
    lease-expiration-duration-in-seconds: 90 # miss 3, you are evicted
```

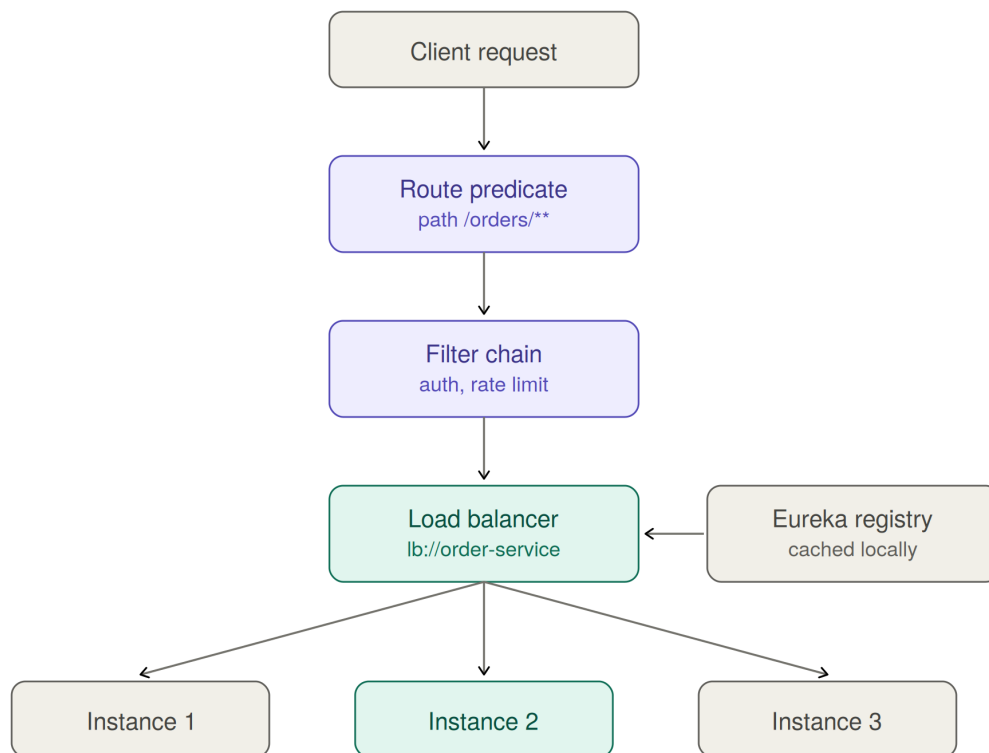
Trap: Eureka is not a load balancer. It is a phone book. Load balancing is client-side — the caller fetches the instance list, caches it, and Spring Cloud LoadBalancer picks one. Eureka never sees your traffic.

3 API gateway

How does the gateway pick an instance?

Predicate matches → filters run → `lb://order-service` hands to Spring Cloud LoadBalancer → **round-robin** pick from the locally cached Eureka list → forward.

Trap: not consistent hashing. That is for sticky sessions and cache-key stability, not routing.



Only the last hop is a network call — everything above it is in-process

The route config

Gateway — application.yml

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-route                # a name, for logs only
          uri: lb://order-service        # ask the list, do not hardcode a host
          predicates:
            - Path=/orders/**           # the "when"
          filters:
            - StripPrefix=1             # chop /orders before forwarding
        - id: user-route
          uri: lb://user-service
          predicates:
            - Path=/users/**
          filters:
            - StripPrefix=1
```

YAML in three rules: key: value; indentation means "belongs to"; a dash is one list item. Tabs break it — spaces only.

Pattern	Matches
/orders	only exactly that
/orders/*	/orders/123 but not /orders/123/items
/orders/**	everything below, any depth

StripPrefix=1: request /orders/123 is forwarded as /123, so the controller stays @GetMapping("/{id}"). The gateway owns the public URL structure; the service owns its own.

Where do predicates and filters actually live?

The gateway is a real Spring Boot app — own repo, own pom.xml, own JAR, deployed on EC2 or ECS. At startup it reads that yml and builds **Java objects in its own heap**: a PathRoutePredicate holding /orders/**, a StripPrefixFilter holding 1. Looping over them is microseconds, no network hop. In production run two or three gateway instances behind an ALB.

4 Config server

Why a config server instead of config in each JAR?

Config lives in Git, so changing a value does not mean rebuilding and redeploying every instance. Services ask the **config server**, not Git — so they need no Git credentials, it caches, and it decrypts secrets.

Config server

```
@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication { }
```

Config server — application.yml

```
server:
  port: 8888

spring:
  cloud:
    config:
      server:
        git:
          uri: https://github.com/satish/config-repo
          default-label: main
```

Each service — application.yml

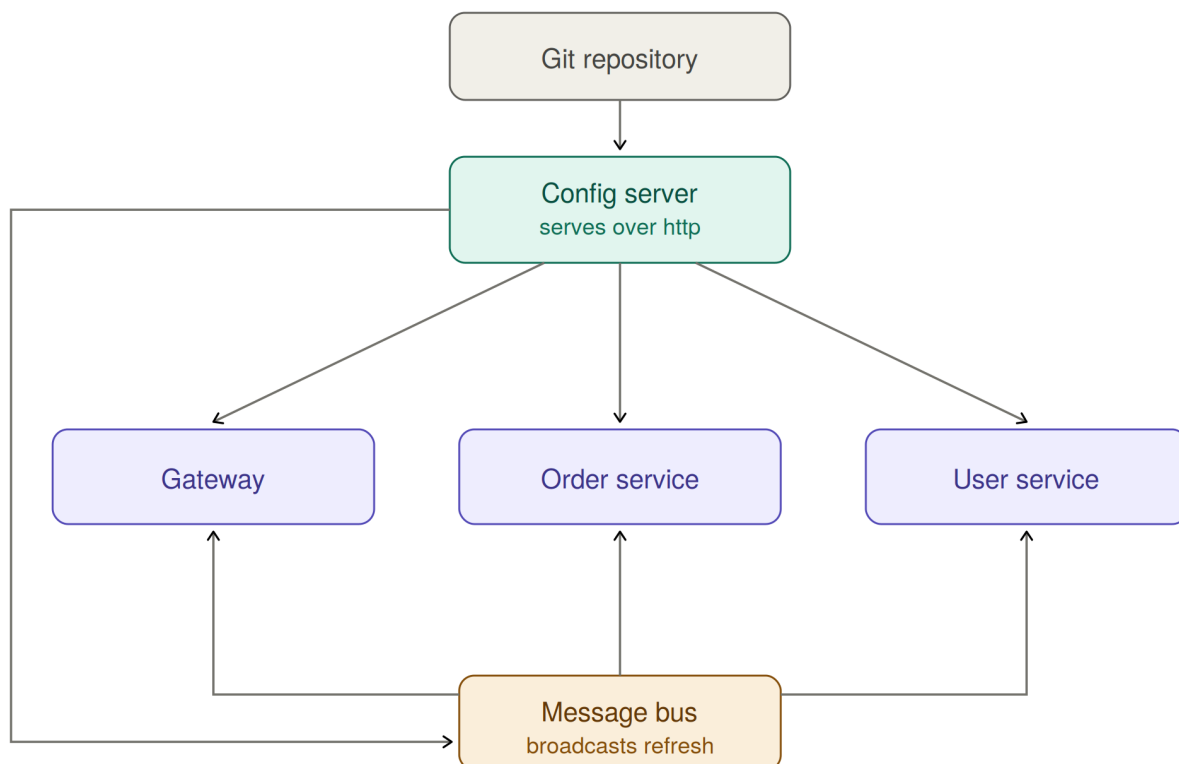
```
spring:
  application:
    name: order-service      # -> order-service.yml in the repo
  profiles:
    active: prod             # -> order-service-prod.yml layered on top
  config:
    import: optional:configserver:http://localhost:8888
```

api-gateway on prod gets api-gateway-prod.yml merged over the shared application.yml.

Trap: services do not poll. Startup is a pull, before the Spring context builds. After that you either call POST /actuator/refresh or run Spring Cloud Bus. Refresh only re-injects @RefreshScope beans and @Value fields — server port and datasources are locked at boot.

What refresh actually reloads

```
@RefreshScope          // without this the bean keeps the old value
@Service
public class PricingService {
    @Value("${pricing.surge-multiplier}")
    private double surge;
}
```



Startup pull downwards; refresh broadcast from the bus back up

Why a message bus instead of calling each service directly?

Direct calls mean the config server must know every running instance and its IP, which changes on every deploy — that is Eureka's job. The bus means one message published once and whoever is listening picks it up. Thirty phone calls versus one announcement over the office speaker. Anything restarting mid-broadcast fetches fresh config at startup anyway.

Spring Cloud Bus — rides on Kafka or RabbitMQ

```
spring:
  cloud:
    bus:
      enabled: true
  kafka:
    bootstrap-servers: localhost:9092

# then one call refreshes every instance:
# POST /actuator/busrefresh
```

5 Adding a new microservice

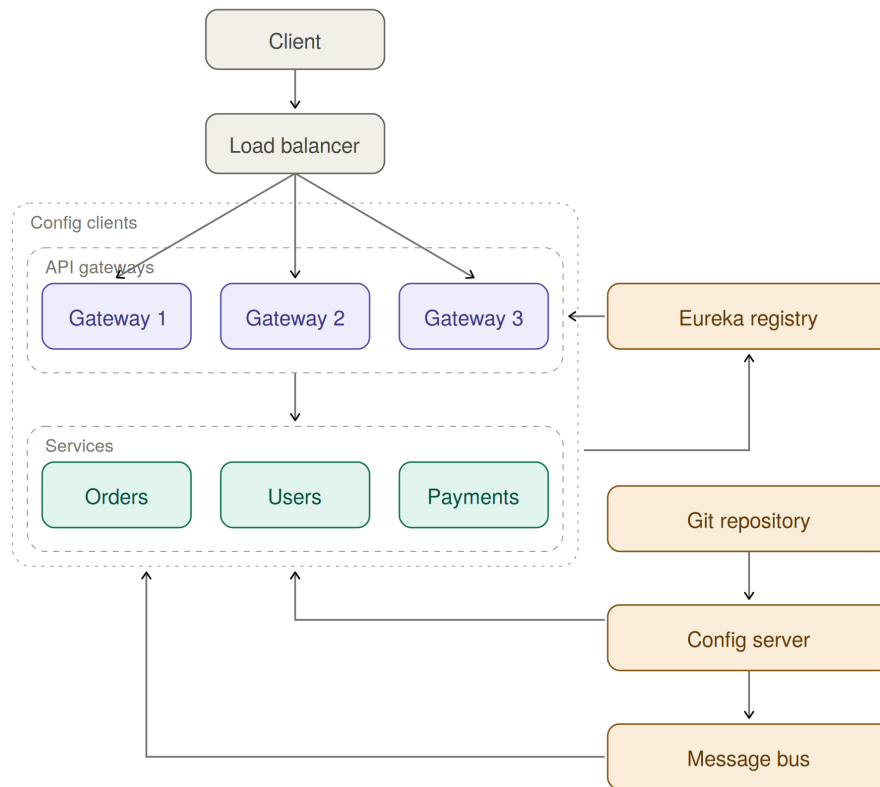
1. Add the eureka-client and config-client dependencies.
2. Set `spring.application.name: inventory-service`.
3. Point it at the config server and at Eureka.
4. Add `inventory-service.yml` to the Git config repo.
5. Add a **route block to the gateway config** in Git.
6. Deploy. On boot: fetch config → register → start heartbeating.
7. Refresh the gateways so they pick up the new route.

Step 5 — the only edit to anything that already exists

```
- id: inventory-route
  uri: lb://inventory-service
  predicates:
    - Path=/inventory/**
```

Untouched: existing services, Eureka's own config, the ALB.

Trap: routes do not go into Eureka — it stores only name, IP, port and health. Miss the route block and the service is registered, healthy and unreachable: 404 at the gateway. One-liner: the service registers itself automatically, but the gateway route is the only manual config change.



Every connection: registration, registry reads, config pull, refresh broadcast

6 Service-to-service calls

Sync or async — how do you decide?

Does a failure here need to block the response? A stock check must be synchronous — you cannot confirm an order without it. A confirmation email must be asynchronous — nobody waits on SMTP, and a mail failure should not fail the order.

RestTemplate vs FeignClient

Both make HTTP calls. The difference is who writes the code.

RestTemplate — you build the call by hand

```
String url = "http://inventory-service/api/stock/" + itemId;
Stock stock = restTemplate.getForObject(url, Stock.class);
// you assemble the URL, pick the method, pass the type – at every call site
```

FeignClient — you declare an interface, Spring writes the implementation

```
@FeignClient(name = "inventory-service")    // resolved via Eureka
public interface InventoryClient {
    @GetMapping("/api/stock/{itemId}")      // same annotations as a controller
    Stock getStock(@PathVariable Long itemId);
}

// call site:
Stock stock = inventoryClient.getStock(itemId);
```

You never wrote an implementation. At startup Spring generates a proxy behind that interface — builds the URL, makes the call, deserialises the response. Same mechanism as `@Transactional`. `name=` resolves through Eureka and round-robins, exactly like the gateway's `lb://`. Resilience4j plugs in with one

annotation.

Enabling it, and adding a circuit-breaker fallback

```
@EnableFeignClients // on the main application class

@FeignClient(name = "inventory-service", fallback = InventoryFallback.class)
```

	Style	Status
RestTemplate	Imperative, blocking	Legacy — maintenance mode since Spring 5
WebClient	Reactive, non-blocking	Current, for direct calls
FeignClient	Declarative interface	Preferred for service-to-service

Saying "RestTemplate is in maintenance mode, WebClient replaced it" is the line that marks you as current rather than reciting a 2018 tutorial.

7 Threading in Spring

Where is the thread pool configured?

Two separate pools — mixing them up is a classic stumble.

1. Your own async work — a @Configuration class

```
@Configuration
@EnableAsync
public class AsyncConfig {

    @Bean(name = "orderExecutor")
    public Executor orderExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(10); // always alive
        executor.setMaxPoolSize(20); // only after the queue is full
        executor.setQueueCapacity(100); // MUST be bounded
        executor.setKeepAliveSeconds(60); // extra threads die after 60s idle
        executor.setThreadNamePrefix("order-");
        executor.initialize();
        return executor;
    }
}
```

Using it — the async email from the Saga discussion

```
@Async("orderExecutor")
public void sendConfirmationEmail(Order order) {
    // runs on a pool thread; the caller returns immediately
}
```

2. Incoming HTTP — you do not write it, you only tune it

```
server:
  tomcat:
    threads:
      max: 200 # max pool
      min-spare: 10 # core pool
    accept-count: 100 # queue
    max-connections: 8192
```

Pool	Who creates it	Handles
Tomcat	The container, at startup	Incoming HTTP requests
Your @Bean	You	@Async background work

Order of admission: threads < core → new thread; else queue not full → queue it; else threads < max → new thread; else reject. The **queue fills before** extra threads are created.

Trap: an unbounded queue makes step two always succeed, so max pool becomes dead config — you sit at 10 threads until OOM. And never call new Thread() in a Spring service.

Beans are singletons — so are they thread-safe?

One instance serves every concurrent request, so any mutable field is shared state. Keep beans stateless; if you must hold a counter, make it atomic.

The bug they look for

```
@Service
public class OrderService {
    private int count = 0;                // BROKEN: count++ is read-add-write
    private final AtomicInteger safe = new AtomicInteger(); // one CAS instruction

    public void handle() {
        count++;                          // lost updates under load
        safe.incrementAndGet();           // correct, no locking
    }
}
```

volatile fixes visibility, not atomicity — private volatile int count with count++ looks thread-safe and is not. Use synchronized for read-then-write, AtomicInteger for simple counters.

8 Design patterns in Spring

Which patterns have you actually used?

Pattern	Where it shows up in your stack
Singleton	Every Spring bean, by default
Proxy	@Transactional, FeignClient, all AOP
Factory	@Bean methods, LoggerFactory
Strategy	Comparator, payment and notification channels
Chain of responsibility	Servlet filters, the gateway filter chain
Template method	JdbcTemplate, RestTemplate — the name is the pattern
Builder	Lombok @Builder, Stream.builder()
Observer	Spring events, Kafka consumers

The Strategy answer worth memorising

```

public interface PaymentProcessor { void pay(Order o); }

@Component("UPI")      class UpiProcessor implements PaymentProcessor { }
@Component("CARD")     class CardProcessor implements PaymentProcessor { }
@Component("NETBANKING") class NbProcessor implements PaymentProcessor { }

@Service
public class PaymentService {
    private final Map<String, PaymentProcessor> processors; // Spring fills this

    public PaymentService(Map<String, PaymentProcessor> processors) {
        this.processors = processors; // keyed by @Component name
    }

    public void pay(String mode, Order order) {
        processors.get(mode).pay(order); // the map lookup IS the factory
    }
}

```

A new processor auto-registers without editing anything. Open/closed in practice. Saying where you would **not** use it — two implementations that will never grow — reads as senior.